

CEUB

EDUCAÇÃO SUPERIOR

Rede de computadores

Aula 08 e 09 – Camada transporte

Prof.: Antônio Carlos

ceub.br

Protocolos Importantes da Camada 3 - Rede

1. IP (Internet Protocol) – Versões IPV4 e IPV6

O IP é o protocolo mais importante da Internet. Sua função é o endereçamento e a fragmentação de pacotes.

- **IPv4:** Utiliza endereços de 32 bits (ex: 192.168.1.1). Devido à escassez de endereços, ele depende muito de tecnologias como o NAT.
- **IPv6:** Criado para substituir o IPv4, utiliza endereços de 128 bits (ex: 2001:0db8:85a3...), oferecendo uma quantidade quase infinita de IPs e maior segurança nativa (IPsec).
- **Característica:** É um protocolo "Best Effort" (Melhor Esforço), ou seja, ele não garante que o pacote chegará; essa responsabilidade é da Camada 4 (TCP).

2. ICMP (Internet Control Message Protocol)

Diferente do IP, o ICMP não transporta dados de usuário. Ele serve para reportar erros e fornecer informações operacionais.

- **Utilidade:** Quando você usa os comandos ping ou tracert no Windows/Linux, você está usando o ICMP.
- **Mensagens Comuns:** "Destination Unreachable" (Destino Inalcançável) ou "Time Exceeded" (quando o TTL do pacote chega a zero).

3. ARP (Address Resolution Protocol)

O ARP é o protocolo "ponte" entre a Camada 3 e a Camada 2. Função: Ele resolve um endereço IP (lógico) em um endereço MAC (físico).

- **Como funciona:** Quando um roteador sabe o IP do próximo salto mas não sabe o MAC da placa de rede dele, ele envia um "ARP Request" em broadcast perguntando: "Quem tem o IP X.X.X.X? Me envie seu MAC".

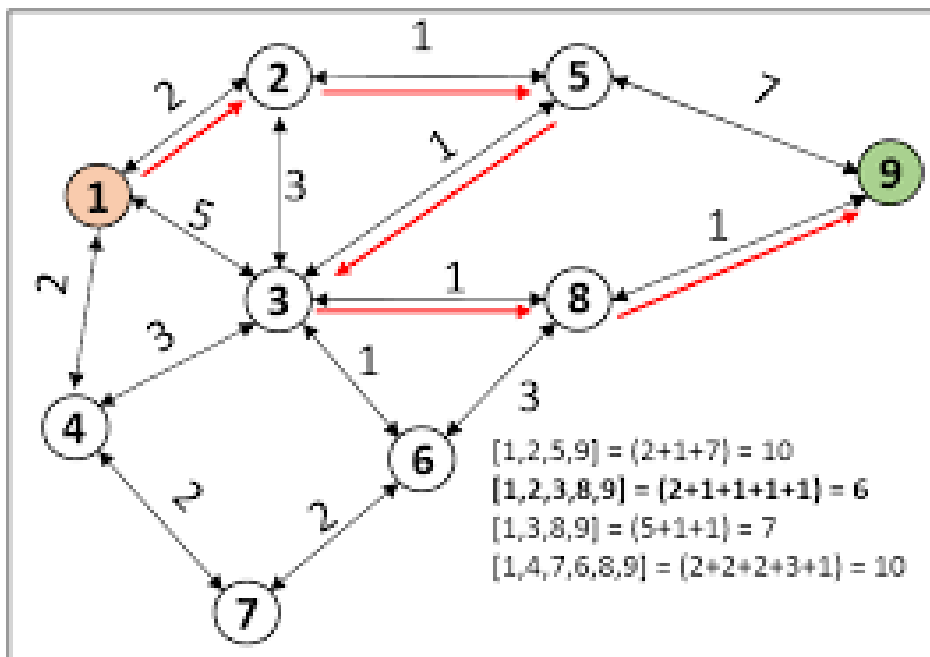
4. Protocolos de Roteamento (Routing Protocols)

Estes protocolos são usados pelos roteadores para construir e manter as Tabelas de Roteamento que você configurou manualmente nos exercícios anteriores.

- **OSPF (Open Shortest Path First):** Um protocolo de estado de link que calcula a rota mais curta usando o algoritmo de Dijkstra. Muito usado em redes corporativas.
- **BGP (Border Gateway Protocol):** O "protocolo da Internet". Ele decide como os pacotes viajam entre diferentes países e provedores (Sistemas Autônomos).
- **RIP (Routing Information Protocol):** Um protocolo mais antigo que usa a contagem de "saltos" (hops) para decidir o caminho.

OSPF (Open Shortest Path First):

O algoritmo de Dijkstra é a inteligência por trás do OSPF. Ele é utilizado para encontrar o caminho mais curto entre um nó de origem (o roteador local) e todos os outros destinos na rede. No contexto de redes, "mais curto" não significa distância física, mas sim o menor custo (métrica).



Edsger Wybe Dijkstra - Holandês

1. A Base de Dados (LSDB)

Antes de rodar o algoritmo, cada roteador OSPF precisa ter um "mapa" completo da rede. Isso é feito através dos LSA (Link State Advertisements). Quando todos os roteadores têm o mesmo mapa, dizemos que a rede convergiu. A partir desse mapa, o roteador se coloca como a "raiz" de uma árvore.

2. O Passo a Passo do Cálculo

O algoritmo de Dijkstra trabalha de forma iterativa:

- **Inicialização:** O roteador marca a si mesmo com custo 0 e todos os outros roteadores como infinito (pois ele ainda não sabe como chegar lá).
- **Exploração de Vizinhos:** O roteador olha para seus vizinhos diretamente conectados e calcula o custo para alcançá-los.

Protocolos da Camada 3 - OSPF

No OSPF, o custo é calculado pela fórmula:

$$Custo = \frac{10^8}{Largura\ de\ banda\ (bps)}$$

Se o roteador usar apenas a fórmula de forma isolada, ele só saberá o custo do "vizinho do lado". O **Dijkstra** é o que permite que ele calcule o custo de uma viagem completa atravessando 10 roteadores diferentes, somando todos os custos individuais no caminho.

3. Seleção do Menor Caminho: O algoritmo escolhe o vizinho com o menor custo acumulado e o marca como "visitado" (nó permanente).

- 4. Atualização de Tabela:** A partir desse novo nó visitado, ele olha para os próximos vizinhos. Se ele encontrar um caminho para um roteador remoto que seja mais barato do que o caminho registrado anteriormente, ele atualiza o valor.
- 5. Repetição:** O processo se repete até que todos os roteadores da área tenham sido visitados e o caminho de menor custo para cada um tenha sido determinado.

Para entender como o Dijkstra toma decisões, vamos imaginar uma topologia de rede e realizar o cálculo passo a passo, como se fôssemos o processador do roteador.

1º Cenário da Rede

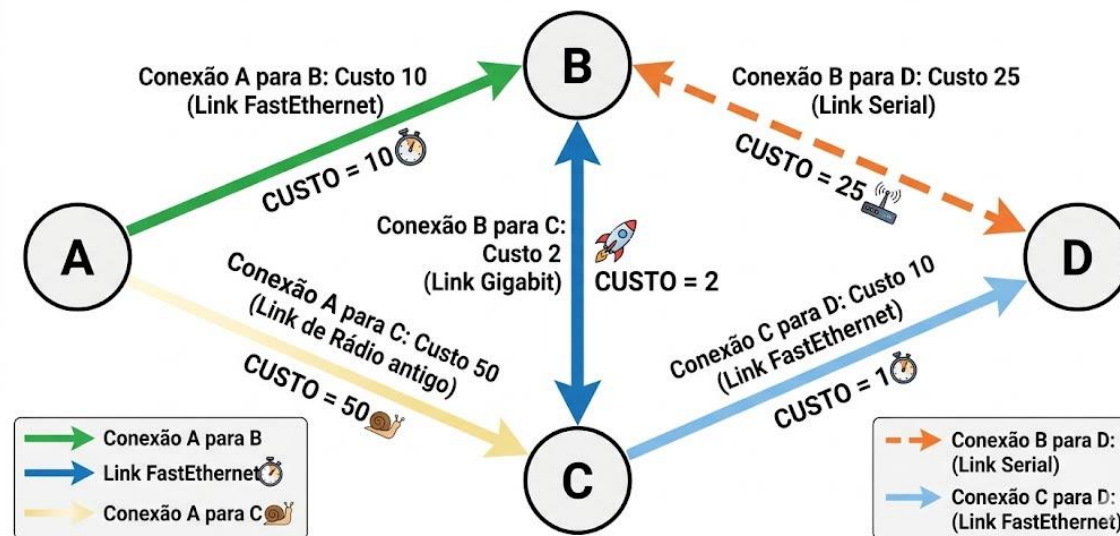
Imagine que o Roteador **A** precisa enviar um pacote para o Roteador **D**. Existem três caminhos possíveis, cada um com uma largura de banda diferente (e, portanto, custos diferentes).

1º Cenário da Rede

Conexões e Custos:

- **A** para **B**: Custo 10 (Link FastEthernet)
- **A** para **C**: Custo 50 (Link de Rádio antigo)
- **B** para **C**: Custo 2 (Link Gigabit)
- **B** para **D**: Custo 25 (Link Serial)
- **C** para **D**: Custo 10 (Link FastEthernet)

VISUALIZAÇÃO DE CUSTOS DE ROTEAMENTO OSPF



Protocolos da Camada 3 - OSPF

Resumo da Lógica "Acumulada"

Destino	Caminho Percorrido	Soma dos Custos	Custo Final
B	$A \rightarrow B$	10	10
C	$A \rightarrow B \rightarrow C$	$10 + 2$	12
D	$A \rightarrow B \rightarrow C \rightarrow D$	$10 + 2 + 10$	22

O valor "Acumulado" é simplesmente a memória do esforço gasto para chegar ao roteador anterior, somada ao novo esforço para atravessar o próximo cabo. É assim que o OSPF garante que não vai apenas pelo caminho "mais curto" em distância, mas pelo caminho que "custa menos" em termos de performance.

2. O Cálculo Passo a Passo (Partindo de A)

O algoritmo mantém uma tabela de "Custo Acumulado". Inicialmente, todos os destinos são infinitos (∞), exceto a origem (A), que é 0.

Passo	Nó Atual	Vizinhos Analisados	Cálculo de Custo Acumulado	Nó Escolhido (Menor Custo)
1	A	B, C	$B = 0+10$ (10) / $C = 0+50$ (50)	B (Custo 10)
2	B	C, D	$C = 10+2$ (12) / $D = 10+25$ (35)	C (Custo 12)
3	C	D	$D = 12+10$ (22)	D (Custo 22)

3. A Decisão da Melhor Rota

Ao final do cálculo, o algoritmo de Dijkstra compara as opções para chegar em D:

Caminho Direto ($A \rightarrow C \rightarrow D$): Custo $50 + 10 = 60$

Caminho Médio ($A \rightarrow B \rightarrow D$): Custo $10 + 25 = 35$

Caminho "Longo" em saltos ($A \rightarrow B \rightarrow C \rightarrow D$): Custo $10 + 2 + 10 = 22$

Veredito: O OSPF escolherá o caminho $A \rightarrow B \rightarrow C \rightarrow D$.

A lógica da rede: Mesmo que esse caminho tenha 3 saltos (passa por mais roteadores), ele utiliza links mais rápidos (Gigabit/FastEthernet). O caminho com menos saltos é descartado porque a "estrada" é muito lenta (Custo 50).

4. Resumo da Tabela de Roteamento do Roteador A

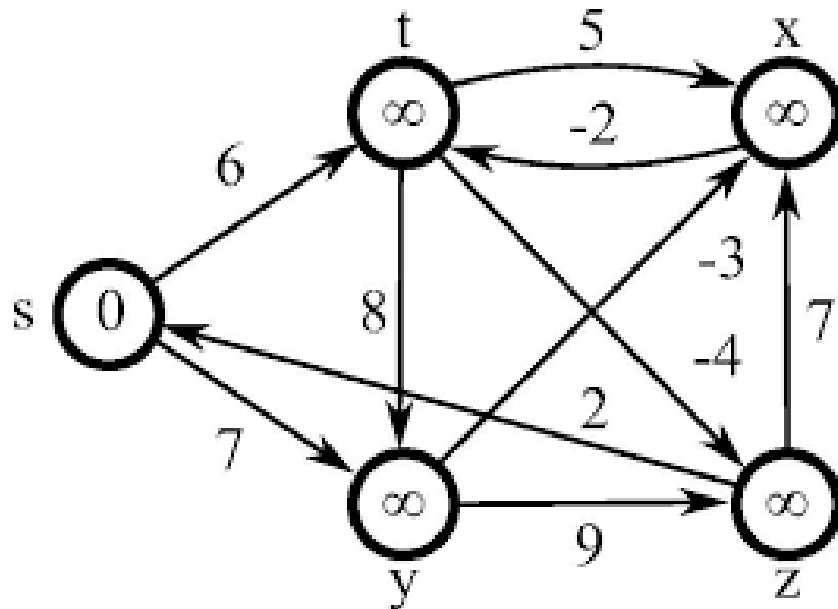
Após o cálculo de Dijkstra, a tabela de roteamento de A será preenchida assim:

Destino	Próximo Salto (Next Hop)	Custo Total
Rede D	Interface ligada ao Roteador B	22

Se o link entre B e C cair, o roteador detecta a mudança via LSA (Link State Advertisement) é a unidade fundamental de informação do protocolo OSPF), o mapa (LSDB - Banco de Dados de Estado de Link) é atualizado e o algoritmo roda novamente, escolhendo instantaneamente o caminho de custo 35 como a nova melhor rota.

Protocolos da Camada 3 - RIP

RIP (Routing Information Protocol) é muito mais simples do que OSPF. Ele utiliza um algoritmo chamado Bellman-Ford e sua métrica é baseada exclusivamente na Contagem de Saltos (Hop Count). Para o RIP, a melhor rota é sempre aquela que tem o menor número de roteadores no caminho, independentemente da velocidade dos cabos.



Bellman & Ford



Richard E. Bellman
(1920-1984)
IEEE Medal of Honor, 1979



Lester R. Ford, Jr.
(1927-)
president of MAA, 1947-48

1. A Métrica: O Salto (Hop)

No RIP, cada roteador que um pacote atravessa conta como 1 salto.

- **Limite Máximo:** O RIP tem um limite de 15 saltos.
- **Rede Inalcançável:** Se um destino estiver a 16 saltos de distância, o RIP considera a rede como "inalcançável" e descarta o pacote. Por isso, ele só é usado em redes pequenas.

2. Como o cálculo é feito (Vetor de Distância)

O cálculo do RIP é conhecido como "Roteamento por Fofoca". Ele funciona assim:

- **Troca de Tabelas:** A cada 30 segundos, o roteador envia sua tabela de roteamento inteira para os seus vizinhos diretos.
- **Soma de Salto:** Quando o Roteador B recebe a tabela do Roteador A, ele pensa: "Se o Roteador A consegue chegar na Rede X com 2 saltos, e eu estou a 1 salto do Roteador A, então eu consigo chegar na Rede X com $2 + 1 = 3$ saltos".

2. Como o cálculo é feito (Vetor de Distância)

Comparação e Decisão:

- Se o Roteador B ainda não conhecia a Rede X, ele a adiciona na tabela com custo 3.
- Se ele já conhecia a Rede X com um custo maior (ex: 5), ele atualiza para o novo caminho mais curto (3).
- Se o novo caminho for mais caro (ex: 6), ele simplesmente ignora.

3. Exemplo de Decisão do RIP

Imagine o mesmo cenário que usamos para o OSPF:

Caminho 1: $A \rightarrow B \rightarrow C \rightarrow D$ (Links de 1 Gbps - Fibra)

Caminho 2: $A \rightarrow E \rightarrow D$ (Links de 10 Mbps - Rádio antigo)

O que o RIP escolheria?

- Caminho 1: Tem 3 saltos.
- Caminho 2: Tem 2 saltos.

O RIP escolheria o Caminho 2, mesmo ele sendo muito mais lento, simplesmente porque tem menos roteadores no meio. **Essa é a principal "fraqueza" do RIP em comparação ao OSPF.**

4. Problemas no Cálculo: Loop de Roteamento

Como o RIP demora para espalhar a informação (fofoca em fofoca), pode ocorrer o problema da "Contagem ao Infinito". Se uma rede cai, dois roteadores podem ficar um dizendo para o outro: "Eu chego lá em 3 saltos", "E eu chego por você em 4", subindo esse valor até atingir o limite de 16.

Para evitar isso, o RIP usa técnicas como:

- **Split Horizon:** Proíbe um roteador de enviar informações sobre uma rota de volta para o mesmo vizinho que lhe enviou essa informação.
- **Poison Reverse:** Quando uma rede cai, o roteador avisa os vizinhos que o custo agora é 16 (infinito) imediatamente.

Protocolos da Camada 3 – Comparativo OSPF e RIP

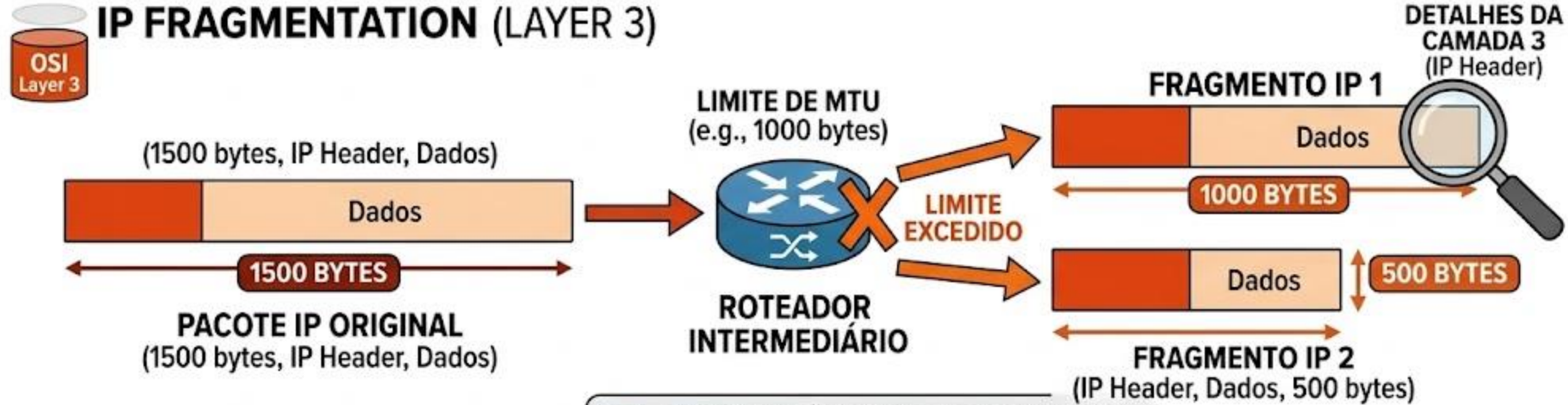
Característica	OSPF (Dijkstra)	RIP (Bellman-Ford)
Métrica	Custo (Baseado em Banda)	Saltos (Hops)
Visão da Rede	Mapa Completo (LSDB)	Apenas vizinhos (Vetor de Distância)
Atualização	Apenas quando há mudanças	A cada 30 segundos (Tabela inteira)
Escalabilidade	Grandes Redes	Pequenas Redes (Máx. 15 saltos)

O RIP é fácil de configurar e consome pouca CPU, mas em redes modernas com velocidades variadas, ele raramente é a melhor escolha.



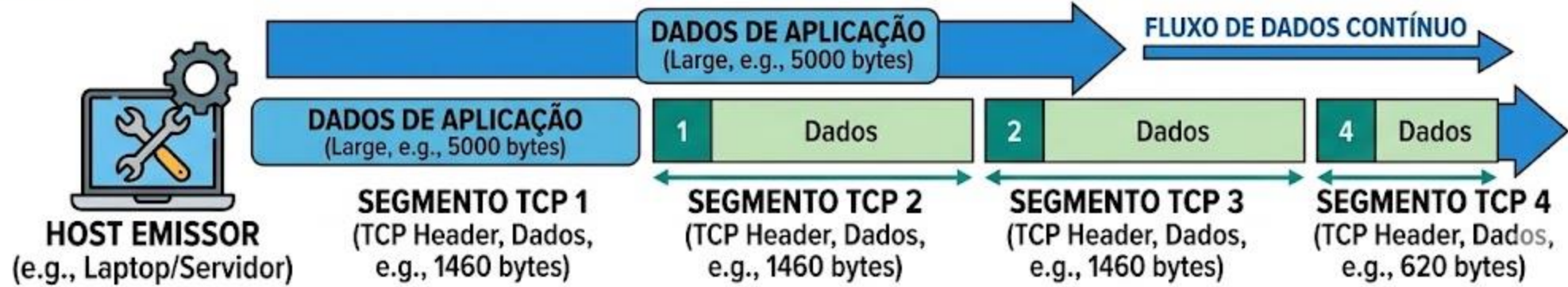
Camada Transporte

Camada 03/04



TCP SEGMENTATION (LAYER 4)

OSI
Layer 4



1. Segmentação (Camada 4 - Transporte)

A segmentação ocorre na Camada de Transporte (geralmente pelo protocolo TCP) no dispositivo de origem (o seu computador, por exemplo).

- **Onde ocorre:** No host de origem.
- **Por que ocorre:** O TCP recebe um fluxo enorme de dados da Camada de Aplicação (como um arquivo de 1 GB) e precisa dividi-lo em pedaços chamados Segmentos.
- **Tamanho:** O tamanho do segmento é determinado pelo MSS (Maximum Segment Size), que é negociado entre as duas pontas da conexão para evitar sobrecarregar a rede.
- **Responsabilidade:** Se um segmento for perdido, o TCP da origem percebe e o reenvia.

2. Fragmentação (Camada 3 - Rede)

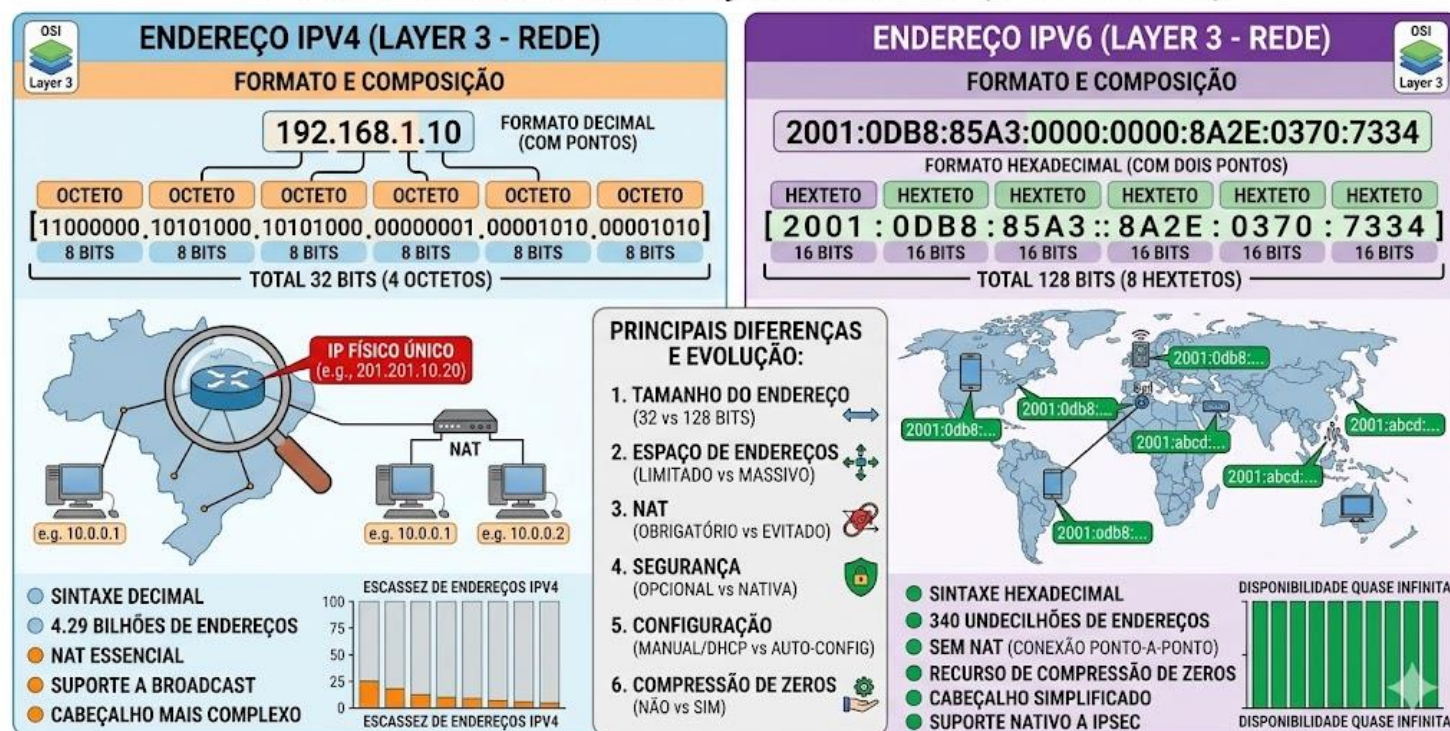
A fragmentação ocorre na Camada de Rede (protocolo IP) e pode ser feita pelo computador de origem ou por qualquer roteador no meio do caminho.

- **Onde ocorre:** Em qualquer nó da rede (roteadores intermediários).
- **Por que ocorre:** Devido ao MTU (Maximum Transmission Unit). Imagine que um roteador recebe um pacote de 1500 bytes, mas a próxima rede (um link antigo ou uma VPN) só suporta pacotes de no máximo 1000 bytes. O roteador é "obrigado" a cortar esse pacote IP em dois ou mais pedaços chamados Fragmentos.
- **Problema:** A fragmentação é ruim para a performance. O roteador gasta CPU para "picar" o pacote e o destino gasta CPU para "colar" tudo de volta. Além disso, se um único fragmento for perdido, o pacote inteiro é descartado.

2. Fragmentação (Camada 3 - Rede)

- **IPv6 vs IPv4:** No IPv4, roteadores podem fragmentar. No IPv6, roteadores não podem fragmentar; se o pacote for grande demais, o roteador o descarta e avisa a origem para enviar algo menor.

ESTRUTURA DE ENDEREÇAMENTO IP (IPv4 E IPv6)



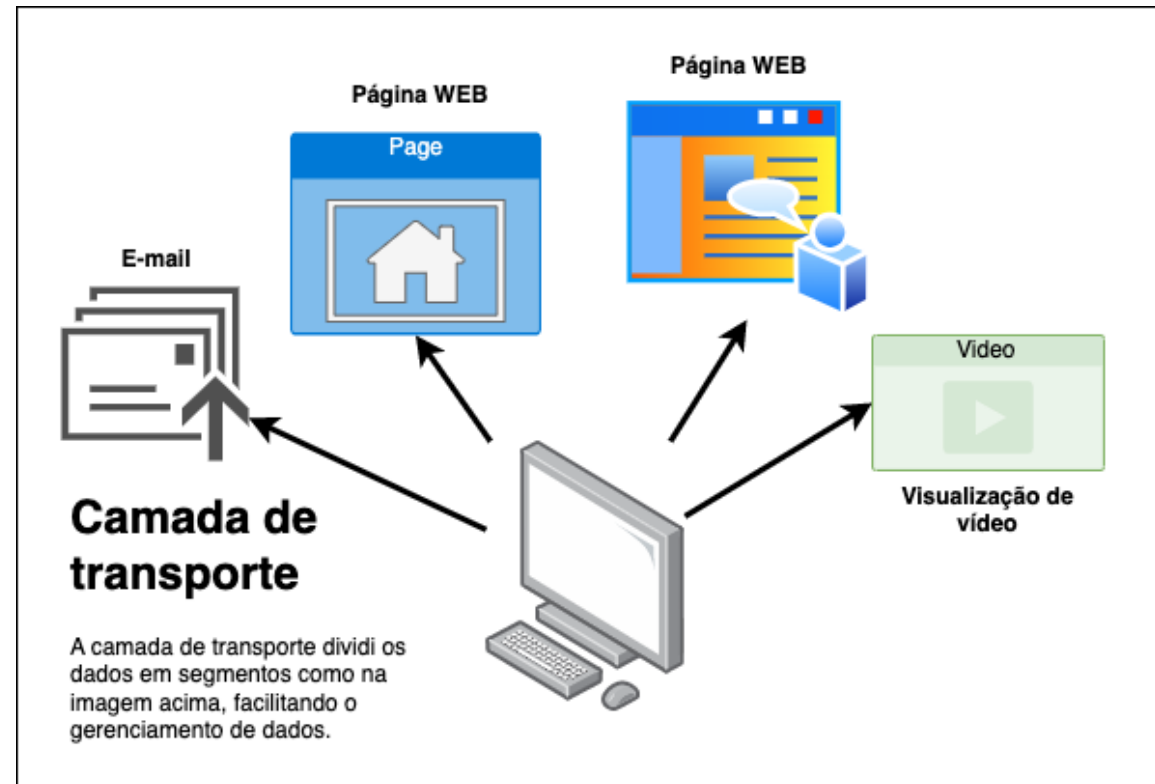
Camada 03/04 - Comparativo

Característica	Segmentação	Fragmentação
Camada OSI	Camada 4 (Transporte)	Camada 3 (Rede)
Protocolo	TCP	IP (v4)
Unidade de Dado	Segmento	Fragmento (do Pacote)
Quem executa	Host de origem (emissor)	Roteadores ou Emissor
Motivo	Organização do fluxo de dados	Limitação física do meio (MTU)

Camada TRANSPORTE

É a quarta camada do Modelo OSI (Open Systems Interconnection). Sua principal função é **fornecer comunicação confiável e eficiente entre hosts em uma rede**, garantindo que os dados sejam transmitidos de maneira correta, independente de erros e da natureza física da rede.

Um **host** é qualquer dispositivo conectado a uma rede que pode oferecer informações, recursos ou serviços a outros dispositivos.



Protocolos Camada TRANSPORTE

TCP (Transmission Control Protocol) - é um protocolo orientado à conexão e confiável, projetado para garantir que os dados sejam entregues corretamente, em ordem, e sem perda ou duplicação.

Transmission Control
Protocol - TCP



TCP (Transmission Control Protocol) - Funcionalidades

Conexão confiável: O TCP estabelece uma conexão lógica entre os dois dispositivos que trocam dados, chamada de "sessão" ou "conexão", através de um processo conhecido como handshake de três vias. Este handshake é realizado antes do início da troca de dados para garantir que ambos os lados estejam prontos para se comunicar.

- **1º SYN (Synchronize):** O cliente envia um pacote com a bandeira SYN para iniciar a conexão.
- **2º SYN-ACK {Synchronize – Acknowledge (Confirmação de recebimento)}:** O servidor responde com um pacote contendo tanto a bandeira SYN quanto ACK.
- **3º ACK (Acknowledge):** O cliente envia uma confirmação ACK, completando a conexão.

TCP (Transmission Control Protocol) - Funcionalidades

Transmissão confiável: O TCP utiliza numeração de sequência e confirmações (ACK) para garantir que cada segmento de dados seja recebido corretamente. Caso um segmento seja perdido ou corrompido, ele será retransmitido.

Controle de fluxo: O TCP ajusta dinamicamente a taxa de envio dos dados para evitar sobrecarregar o destinatário ou a rede. Isso é feito por meio de janelas deslizantes e do tamanho da janela de recepção.

TCP (Transmission Control Protocol) - Funcionalidades

Controle de congestionamento: O TCP também implementa mecanismos para lidar com a congestão da rede, como a partida lenta (slow start), o controle de congestionamento e a evitação de congestionamento. Esses mecanismos ajustam a taxa de transmissão dos dados com base nas condições atuais da rede.

Entrega ordenada: O TCP garante que os dados cheguem na ordem correta, utilizando números de sequência e reordenando os pacotes no destino.

Segmentação e Reagregação: O TCP divide grandes blocos de dados em segmentos menores para transmissão e, ao recebê-los, os reagrupa na ordem correta.

TCP (Transmission Control Protocol) - Uso

Aplicações que exigem alta confiabilidade e controle rigoroso sobre a entrega dos dados, como navegação web (HTTP/HTTPS), transferência de arquivos (FTP), envio de e-mails (SMTP) e acesso remoto (SSH).

Características

- Confiável: Sim.
- Orientado a conexão: Sim. (se conecta antes de enviar o pacote)
- Controle de fluxo: Sim.
- Controle de congestionamento: Sim.
- Correção de erros: Sim (retransmissão de pacotes).

Comunicação

- Só se comunica através de comunicação **unicast** (um-para-um)

Protocolos Camada TRANSPORTE

UDP (User Datagram Protocol)

O UDP é um protocolo **não orientado à conexão e não confiável**, usado quando a velocidade é mais importante do que a precisão ou a confiabilidade da transmissão.

USER DATAGRAM PROTOCOL (UDP)



UDP (User Datagram Protocol) - Funcionalidades

Sem conexão: O UDP não estabelece uma conexão formal antes de enviar os dados. Em vez disso, ele simplesmente envia datagramas (unidades de dados) para o destino. Isso significa que não há handshakes ou confirmações.

Baixa sobrecarga: O UDP possui cabeçalhos muito menores do que o TCP, o que o torna ideal para situações em que a **latência e o desempenho são mais importantes do que a confiabilidade**, como em transmissões de vídeo em tempo real, voz sobre IP (VoIP), jogos online e streaming de mídia.

UDP (User Datagram Protocol) - Funcionalidades

Entrega não garantida: Não há garantias de que os dados chegarão ao destino ou que chegarão na ordem correta. Se um pacote for perdido, ele não será retransmitido automaticamente, o que pode resultar em perda de dados em transmissões de vídeo, áudio ou outros serviços de tempo real.

Sem controle de fluxo ou controle de congestionamento: O UDP não regula a taxa de envio de dados, o que o torna vulnerável a problemas de congestionamento em redes sobrecarregadas. Cabe à aplicação cuidar de qualquer controle necessário.

UDP (User Datagram Protocol) - Uso

Aplicações que precisam de baixa latência e podem tolerar perda de pacotes, como jogos online, streaming de vídeo, chamadas VoIP e transmissões de vídeo ao vivo.

Características

- **Confiável:** Não.
- **Orientado a conexão:** Não.
- **Controle de fluxo:** Não.
- **Controle de congestionamento:** Não.
- **Correção de erros:** Não.

Comunicação

- É usado em broadcast (para todos), unicast (um-para-um) e multicast (mais de um – grupo)

Protocolos Camada TRANSPORTE

Comparação entre TCP e UDP

Característica	TCP	UDP
Confiabilidade	Confiável (garantia de entrega)	Não confiável (sem garantia de entrega)
Controle de fluxo	Sim	Não
Controle de erros	Sim (retransmissão em caso de erro)	Não
Orientado a conexão	Sim (handshake de 3 vias)	Não (sem handshake)
Overhead	Alto (mais cabeçalhos e controle)	Baixo (menos sobrecarga de cabeçalhos)
Uso típico	HTTP, FTP, SMTP e SSH	VoIP, streaming de vídeo, jogos online

Protocolos Camada TRANSPORTE

Outros Protocolos da Camada de Transporte

Embora TCP e UDP sejam os protocolos mais utilizados, há outros protocolos específicos que também operam na camada de transporte.

SCTP (Stream Control Transmission Protocol):

- Um protocolo de transporte relativamente novo, que combina recursos do TCP e UDP.
- Fornece conexão confiável como o TCP, mas suporta o envio de múltiplos fluxos independentes dentro de uma mesma conexão.
- Utilizado em aplicações que necessitam de robustez em comunicação, como sinalização de telecomunicações e voz sobre IP.

DCCP (Datagram Congestion Control Protocol):

- Protocolo sem conexão, mas que inclui controle de congestionamento.
- Projetado para aplicações em tempo real, que precisam de controle de congestionamento sem o overhead completo de um protocolo orientado a conexão, como TCP.

Protocolos Camada TRANSPORTE

A escolha entre os protocolos de transporte depende das necessidades específicas da aplicação. **Para aplicações que exigem alta confiabilidade e entrega ordenada, como o envio de e-mails ou transferência de arquivos, o TCP é a escolha ideal.** Por outro lado, para **transmissões em tempo real, onde a latência é mais crítica do que a confiabilidade absoluta, o UDP é o mais adequado.** Ambos os protocolos desempenham papéis essenciais na comunicação em redes de computadores e garantem que as aplicações possam se comunicar de forma eficiente e de acordo com suas necessidades.

Principais Funções da Camada de Transporte

1º Segmentação e Reagregação de Dados

- Quando uma aplicação deseja enviar dados, esses dados são muito grandes para serem enviados como um único bloco. **A camada de transporte divide os dados em pedaços menores chamados de segmentos (ou datagramas, dependendo do protocolo).**
- Ao receber os segmentos, a camada de transporte no destinatário os reagrega de volta na ordem correta, garantindo que a mensagem original seja reconstruída adequadamente.

Principais Funções da Camada de Transporte

2º Controle de Fluxo

- A camada de transporte regula a quantidade de dados que o emissor pode enviar antes de receber uma confirmação do destinatário. Esse processo evita sobrecarregar o destinatário com muitos dados de uma vez.
- O controle de fluxo é implementado utilizando janelas deslizantes (sliding windows), onde o remetente envia dados e, com base em confirmações recebidas, ajusta a quantidade de dados que pode continuar enviando.

Principais Funções da Camada de Transporte

3º Controle de Erros

- A camada de transporte oferece mecanismos para detecção de erros nos dados transmitidos. Isso é feito através de códigos de verificação (checksums) que são incluídos em cada segmento.
- Se um erro for detectado, o segmento é descartado e o remetente é solicitado a retransmitir o segmento.

Principais Funções da Camada de Transporte

4º Multiplexação e Demultiplexação

- Em uma rede, vários aplicativos podem estar transmitindo dados simultaneamente. A camada de transporte permite que várias comunicações independentes aconteçam ao mesmo tempo, usando portas para identificar quais dados pertencem a qual aplicação.
- **A multiplexação permite que múltiplos fluxos de dados sejam enviados através de uma única conexão. A demultiplexação ocorre quando a camada de transporte identifica e separa os dados recebidos, enviando-os para as aplicações corretas.**

Camada TRANSPORTE

Principais Funções da Camada de Transporte

5º Controle de Conexão: Orientada a Conexão ou Sem Conexão

- Orientada a conexão (TCP - Transmission Control Protocol): **Antes de transferir dados, a camada de transporte estabelece uma conexão confiável entre o emissor e o receptor.** Esse processo é chamado de handshake de três vias, que garante que ambos os lados estejam prontos para a comunicação.
- Não orientada a conexão (UDP - User Datagram Protocol): **Aqui, os dados são enviados sem estabelecer uma conexão prévia.** O UDP é mais rápido, mas não garante a entrega confiável dos dados, sendo usado em situações onde a velocidade é mais importante que a confiabilidade (como transmissão de vídeo em tempo real).

Tamanho dos Pacotes (Datagramas)

- Tamanho dos pacotes em TCP e UDP não é uma característica fixa ou comparável diretamente. **O tamanho de um pacote (ou segmento no TCP e datagrama no UDP) depende da aplicação e da configuração da rede, não do protocolo em si.**
- O tamanho máximo de um pacote enviado, em qualquer dos protocolos, **é limitado pelo MTU (Maximum Transmission Unit) da rede**, e ambos podem gerar pacotes com tamanhos variáveis, dependendo das circunstâncias.

Portanto, o UDP não usa pacotes "menores" que o TCP de forma inerente. Os pacotes do UDP podem ser grandes ou pequenos, dependendo do que a aplicação está enviando e da MTU da rede.

Velocidade e Eficiência

O UDP é considerado mais rápido do que o TCP, **não porque usa pacotes menores**, mas por conta de sua simplicidade:

- **Menor sobrecarga** - Como o UDP não precisa estabelecer uma conexão nem verificar se os pacotes chegaram ou se estão na ordem correta, ele pode enviar dados de forma contínua e sem interrupções, o que resulta em menor latência.
- O TCP, **por outro lado, adiciona uma série de recursos que aumentam a confiabilidade e o controle da transmissão, o que leva a maior latência** devido à retransmissão de pacotes perdidos e ao controle de congestionamento.

Quando o UDP é mais rápido?

O UDP é mais rápido em aplicações que podem tolerar a perda de alguns pacotes, como transmissões ao vivo, jogos online ou chamadas de voz/vídeo. Nesses casos, a latência (tempo de resposta) é mais importante que a precisão da transmissão.

O TCP é mais **lento em comparação porque implementa mecanismos de correção de erros e controle de fluxo**, o que o torna mais confiável, mas menos eficiente em termos de latência.

Camada TRANSPORTE

O que o UDP faz na transmissão de vídeos?

O UDP (User Datagram Protocol) é **amplamente utilizado em aplicações de streaming de vídeo em tempo real** (como YouTube, Netflix, transmissões ao vivo, etc.) porque, diferentemente do TCP, ele não realiza o controle de fluxo ou garantia de entrega. Isso significa que o UDP envia pacotes de dados continuamente, sem verificar se eles chegaram ao destino ou se estão na ordem correta. Para aplicações de vídeo e áudio em tempo real, isso é vantajoso porque atrasos (latência) e retransmissões podem prejudicar a experiência do usuário.



Pacotes Menores?

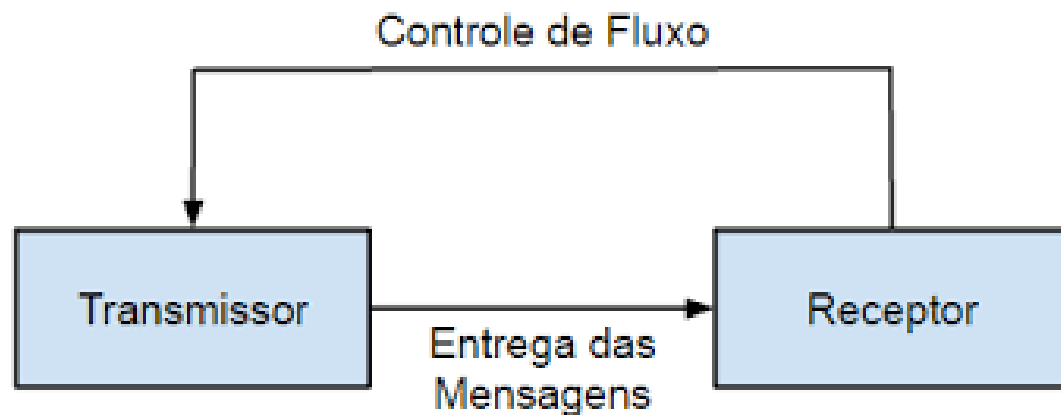
O tamanho dos pacotes que o UDP transmite não é uma característica exclusiva do protocolo em si, mas sim das aplicações e das necessidades da transmissão de dados. **O tamanho dos pacotes é determinado pela aplicação** (por exemplo, o servidor de vídeo) e pela MTU (Unidade Máxima de Transmissão) da rede, não pelo protocolo de transporte.

O UDP pode enviar pacotes de tamanhos variados, e esses pacotes podem ser fragmentados, se necessário, pela camada de rede (IP). Entretanto, o fato de o UDP ser "mais rápido" para certas aplicações não está relacionado ao tamanho dos pacotes, mas sim à ausência de certos mecanismos de controle e verificação que o TCP possui.

Camada TRANSPORTE

Controle de FLUXO

Na camada transporte é um mecanismo que garante que o transmissor (quem está enviando os dados) não sobrecarregue o receptor (quem está recebendo os dados) com mais dados do que ele pode processar. Esse mecanismo evita perda de pacotes, congestionamento e melhora a eficiência da comunicação.



O controle de fluxo ocorre principalmente no **TCP**, já que o **UDP** não oferece controle de fluxo.

Controle de FLUXO

1º Estabelecimento da Conexão (Three-Way Handshake):

- Antes que qualquer dado seja transmitido, o TCP usa o processo de Three-Way Handshake para estabelecer uma conexão confiável entre o transmissor e o receptor. Durante este processo, os dois dispositivos trocam informações, incluindo o **tamanho da janela de recepção** (também chamada de janela de controle de fluxo).
- **A janela de recepção define a quantidade máxima de dados que o receptor pode receber de uma só vez** antes de precisar confirmar que os dados foram recebidos corretamente.

Controle de FLUXO

2º Envio de Dados em Segmentos (TCP Segments):

- O transmissor começa a enviar dados ao receptor em unidades chamadas segmentos TCP.
- O transmissor controla o número de segmentos enviados com base no tamanho da janela de recepção que o receptor informou no handshake inicial. Isso significa que o transmissor só enviará a quantidade de dados que o receptor é capaz de processar.

Controle de FLUXO

3º Recepção de Dados:

- O receptor começa a receber os segmentos de dados.
- Após receber os dados, **o receptor envia mensagens de reconhecimento (ACKs)** ao transmissor, confirmando que os dados foram recebidos corretamente e que está pronto para receber mais.

Controle de FLUXO

4º Ajuste Dinâmico da Janela de Controle de Fluxo:

- **O receptor tem a capacidade de ajustar dinamicamente o tamanho da janela de recepção.** Se o receptor perceber que está ficando sobrecarregado (por exemplo, o buffer de recepção está ficando cheio), ele pode reduzir o tamanho da janela de recepção no próximo ACK enviado ao transmissor.
- Da mesma forma, **se o receptor perceber que está processando os dados rapidamente e pode receber mais, ele pode aumentar o tamanho da janela de recepção.**

Controle de FLUXO

5º Envio de Dados Limitado pela Janela de Controle de Fluxo:

- O transmissor respeita o tamanho da janela de recepção informada pelo receptor.
- **Por exemplo, se o receptor disser que a janela de recepção é de 10 KB, o transmissor não enviará mais que 10 KB de dados antes de receber uma confirmação do receptor.**
Se o receptor diminuir a janela, o transmissor enviará menos dados; se o receptor aumentar a janela, o transmissor enviará mais dados.

Controle de FLUXO

6º Envio de Reconhecimentos (ACKs):

- **O transmissor e o receptor trocam constantemente ACKs para garantir que os dados foram recebidos corretamente.** Cada ACK inclui o número de sequência do último segmento de dados recebido com sucesso e o tamanho atual da janela de recepção.
- **Se o receptor não enviar um ACK dentro de um determinado tempo (timeout), o transmissor entende que houve algum problema,** como perda de pacotes, e retransmite os segmentos não reconhecidos.

Controle de FLUXO

7º Ajuste de Fluxo para Evitar Congestionamento:

- Se o receptor não consegue processar os dados com rapidez suficiente, **ele pode informar uma janela zero**, indicando ao transmissor para pausar o envio de novos dados até que ele esteja pronto para receber novamente.
- **Quando o receptor estiver pronto para continuar, ele envia uma atualização com o novo tamanho da janela de recepção**, permitindo que o transmissor continue enviando os dados.

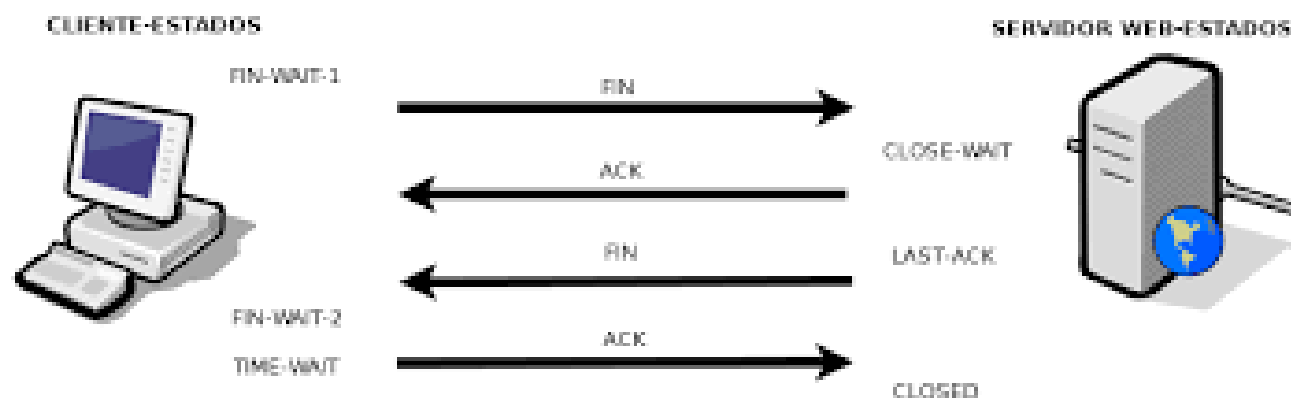
Camada TRANSPORTE

Controle de FLUXO

8º Finalização da Conexão:

Uma vez que todos os dados tenham sido enviados e confirmados, o TCP finaliza a conexão com uma troca de mensagens de término (four-way handshake).

TCP FECHAMENTO DE CONEXÃO



Controle de FLUXO

Importância do Controle de Fluxo:

Evita Perda de Dados: O controle de fluxo evita que o receptor fique sobrecarregado e perca pacotes, garantindo que ele receba os dados no seu próprio ritmo.

Otimiza o Desempenho: O ajuste dinâmico da janela de recepção permite que a rede se ajuste conforme o congestionamento ou capacidade do receptor, otimizando o desempenho.

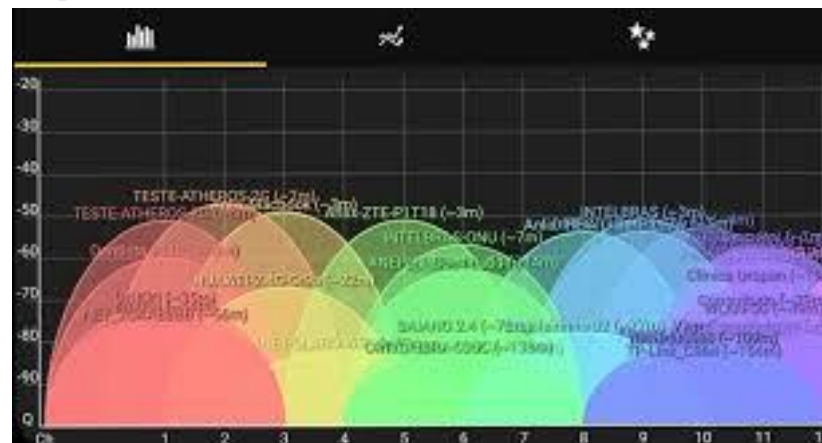
Melhora a Confiabilidade: O controle de fluxo faz parte do conjunto de mecanismos que torna o TCP um protocolo confiável e adequado para aplicações que requerem entrega garantida de dados.

Camada TRANSPORTE

Controle de FLUXO

Controle de Fluxo no UDP

O UDP não implementa controle de fluxo, o que significa que o transmissor pode enviar quantos dados quiser sem verificar a capacidade de processamento do receptor. Isso pode levar à perda de pacotes ou congestionamento na rede se o transmissor enviar mais dados do que o receptor pode lidar.



Camada TRANSPORTE

Controle de Congestionamento

O controle de congestionamento na camada de transporte tem como **objetivo evitar que a rede se sobrecarregue com uma quantidade excessiva de pacotes**, o que poderia resultar em quedas de desempenho, perdas de pacotes e aumento de latência. **Esse mecanismo é fundamental no TCP (Transmission Control Protocol), pois o UDP (User Datagram Protocol), por exemplo, não implementa controle de congestionamento.**

Controle de Congestionamento

1º Janela de Congestionamento (Congestion Window - cwnd):

O TCP utiliza um conceito chamado janela de congestionamento (cwnd). **Essa janela controla quantos dados podem ser enviados sem receber uma confirmação de recebimento (ACK) do destinatário.** No início de uma conexão, o valor de cwnd começa pequeno e é ajustado ao longo do tempo conforme a rede responde à transmissão dos dados.

Controle de Congestionamento

2º Fase de "Partida Lenta" (Slow Start):

O TCP começa enviando poucos pacotes para não sobrecarregar a rede logo de início.

- Inicialmente, o valor da janela de congestionamento é pequeno (geralmente 1 ou 2 segmentos TCP). A cada vez que um ACK (confirmação de recebimento) é recebido pelo remetente, o cwnd é duplicado (ou seja, a janela de congestionamento cresce exponencialmente). Isso continua até atingir um limiar de congestionamento ou até detectar perda de pacotes.
- O objetivo da partida lenta é aumentar a taxa de envio de dados progressivamente até que a rede mostre sinais de congestionamento (geralmente identificados por pacotes perdidos ou ACKs demorados).

Controle de Congestionamento

3º Detecção de Congestionamento:

- Um pacote não é confirmado no tempo esperado (o TCP não recebe um ACK para aquele pacote), o que pode ocorrer quando um roteador na rede não conseguiu encaminhá-lo devido ao congestionamento.
- TCP recebe múltiplos ACKs duplicados para o mesmo pacote (isso pode indicar que o destinatário recebeu pacotes fora de ordem, possivelmente devido à perda de pacotes intermediários).

Quando o congestionamento é detectado, o TCP ajusta seu comportamento.

Controle de Congestionamento

4º Ajustes de Janela de Congestionamento:

Após a detecção de congestionamento, o TCP ajusta o tamanho da janela de congestionamento para controlar a taxa de envio de pacotes e reduzir o congestionamento.

Algoritmo de Evitar Congestionamento (Congestion Avoidance):

Quando o TCP detecta o congestionamento, ele muda do crescimento exponencial da fase de slow start para um crescimento mais controlado e linear.

O tamanho da cwnd é aumentado em um segmento a cada intervalo de tempo de retorno de ACKs (RTT - Round Trip Time). Isso evita que a taxa de envio de pacotes cresça rapidamente e sobrecarregue a rede.

Controle de Congestionamento

5º Treshold (SSThresh - Slow Start Threshold):

O SSThresh é o valor limite usado pelo TCP para decidir quando alternar entre a fase de "partida lenta" e a fase de "evitação de congestionamento".

- Se o cwnd for menor que o SSThresh, o TCP estará na fase de slow start, o que significa que a janela de congestionamento está crescendo exponencialmente.
- Se o cwnd for maior que o SSThresh, o TCP estará na fase de evitação de congestionamento, onde o crescimento da janela será linear e controlado.

Após um evento de congestionamento (perda de pacotes), o valor de SSThresh é ajustado para a metade do valor do cwnd no momento em que o congestionamento foi detectado, para que a próxima vez que a rede se recupere, o TCP saiba como ajustar o crescimento da janela de congestionamento.

Controle de Congestionamento

6º Fast Retransmit e Fast Recovery

Além do controle de congestionamento padrão, o TCP possui mecanismos adicionais para lidar com o congestionamento de forma mais eficiente:

- **Fast Retransmit:** Quando o remetente recebe três ACKs duplicados (sugerindo que um pacote foi perdido), o TCP retransmite imediatamente o pacote perdido sem esperar o tempo limite (timeout). Isso ajuda a corrigir perdas de pacotes de forma mais rápida.
- **Fast Recovery:** Após o fast retransmit, o TCP não retorna imediatamente à fase de partida lenta. Em vez disso, ele entra no fast recovery, onde o valor de cwnd é ajustado para evitar congestionamento, mas sem reduzir a taxa de transmissão tão drasticamente quanto na multiplicative decrease.

Controle de Congestionamento

7º Recuperação Progressiva (Linear Growth)

Após um evento de congestionamento, o TCP continua a operar no modo de crescimento linear (evitação de congestionamento), aumentando o tamanho da janela de congestionamento lentamente até que atinja o limite do $SSThresh$. Isso ajuda a recuperar gradualmente a taxa de transmissão e reduzir o risco de congestionamento.

Fontes

- **TANENBAUM**, A. S. Redes de computadores. 4ª edição. Rio de Janeiro, RJ: Campus, 2003.
- **FOROUZAN**, B. A. Comunicação de dados e redes de computadores. 4a edição. McGraw- Hill, 2008.
- **KUROSE**, J. F.; **ROSS**, K. W. Redes de computadores e a Internet. 5ª edição. São Paulo, SP: Pearson, 2010.
- <https://www.cisco.com/site/us/en/products/security/firewalls/index.html>
- <https://www.paloaltonetworks.com/>

